

21.5-AI-ASS 2303A52249 A.VAMSHI KRISHNA

TASK-1

Prompt: AI-Based Initial Program Development

Use AI to generate a basic program (e.g., Student Grade Calculator).

Ask AI to enhance it with validation and additional features. Compare both versions.

Code & Output:

The top screenshot shows the initial code for the Student Grade Calculator. It includes a function to validate marks and a function to calculate the average and grade. The code is written in Python and is saved as `week11_task_a_student_grade_calculator.py`.

```
def validate_marks(marks: Iterable[float]) -> List[float]:
    validated_marks = list(marks)
    if not validated_marks:
        raise ValueError("At least one mark is required.")

    for mark in validated_marks:
        if not isinstance(mark, (int, float)):
            raise TypeError("Marks must be numeric values.")
        if mark < 0 or mark > 100:
            raise ValueError("Marks must be between 0 and 100.")

    return validated_marks

def enhanced_student_grade_calculator(student_name: str, marks: Iterable[float]) -> GradeResult:
    """Return a validated grade summary with remarks and pass/fail status."""
    cleaned_name = validate_name(student_name)
    validated_marks = validate_marks(marks)
    average = sum(validated_marks) / len(validated_marks)

    if average >= 90:
        grade = "A"
        remarks = "Excellent performance"
    elif average >= 80:
        grade = "B"
        remarks = "Very good performance"
    elif average >= 70:
        grade = "C"
        remarks = "Good performance"
    elif average >= 60:
        grade = "D"
        remarks = "Needs improvement"
    else:
        grade = "F"
        remarks = "Fail"

    return GradeResult(
        student_name=cleaned_name,
        average=average,
        grade=grade,
        passed=average >= 60,
        remarks=remarks,
    )
```

The bottom screenshot shows the enhanced code with more detailed validation and a terminal output showing the program's execution. The code is written in Python and is saved as `week11_task_a_student_grade_calculator.py`.

```
def validate_marks(marks: Iterable[float]) -> List[float]:
    validated_marks = list(marks)
    if not validated_marks:
        raise ValueError("At least one mark is required.")

    for mark in validated_marks:
        if not isinstance(mark, (int, float)):
            raise TypeError("Marks must be numeric values.")
        if mark < 0 or mark > 100:
            raise ValueError("Marks must be between 0 and 100.")

    return validated_marks

def enhanced_student_grade_calculator(student_name: str, marks: Iterable[float]) -> GradeResult:
    """Return a validated grade summary with remarks and pass/fail status."""
    cleaned_name = validate_name(student_name)
    validated_marks = validate_marks(marks)
    average = sum(validated_marks) / len(validated_marks)

    if average >= 90:
        grade = "A"
        remarks = "Excellent performance"
    elif average >= 80:
        grade = "B"
        remarks = "Very good performance"
    elif average >= 70:
        grade = "C"
        remarks = "Good performance"
    elif average >= 60:
        grade = "D"
        remarks = "Needs improvement"
    else:
        grade = "F"
        remarks = "Fail"

    return GradeResult(
        student_name=cleaned_name,
        average=average,
        grade=grade,
        passed=average >= 60,
        remarks=remarks,
    )
```

The terminal output shows the program's execution. It prompts the user to enter the student name and marks, and then displays the calculated average, grade, and remarks.

```
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code> python week11_task_a_student_grade_calculator.py
Student: kishna
Marks: 40
Total: 40.00
This version accepts user input for the student name and marks.
It validates the entered data, calculates total marks, average, and grade,
and shows pass/fail remarks for the final result.
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code> python week11_task_a_student_grade_calculator.py
Enter student details below
=====
WEEK 11 - TASK A: STUDENT GRADE CALCULATOR
=====
Enter student name: kishna
Enter marks separated by spaces: 40
=====
RESULT
Student: kishna
Marks: 40
Total: 40.00
Average: 40.00
Grade: F
Passed: No
Remarks: Fail
=====
SUMMARY
=====
```

Observation:

The AI-generated initial program provided a quick and correct baseline implementation. After enhancement, the program included input validation and additional features, improving reliability and usability. The comparison showed that AI significantly helps in refining logic and reducing development time.

TASK-2

Prompt: AI-Assisted Application Improvement

Generate a simple program (e.g., Library Management).

Use AI to add new functionalities.

Analyze improvements in structure and performance.

Code & Output:

The top screenshot shows the VS Code editor with the file explorer on the left, the editor window in the center, and the chat window on the right. The editor window displays the code for the library management system, including the `analyze_improvements()` function and the `run_improved_demo()` function. The chat window shows the AI's response to the prompt, providing a detailed analysis of the improvements and a list of recommended changes.

The bottom screenshot shows the VS Code editor with the file explorer on the left, the editor window in the center, and the chat window on the right. The editor window displays the output of the program, showing the results of the `run_improved_demo()` function. The chat window shows the AI's response to the prompt, providing a detailed analysis of the improvements and a list of recommended changes.

Observation:

AI-assisted modifications enhanced the structure and functionality of the application. New features were added efficiently, and the code became more modular and readable. Performance and maintainability improved due to better organization and optimized logic.

TASK-3

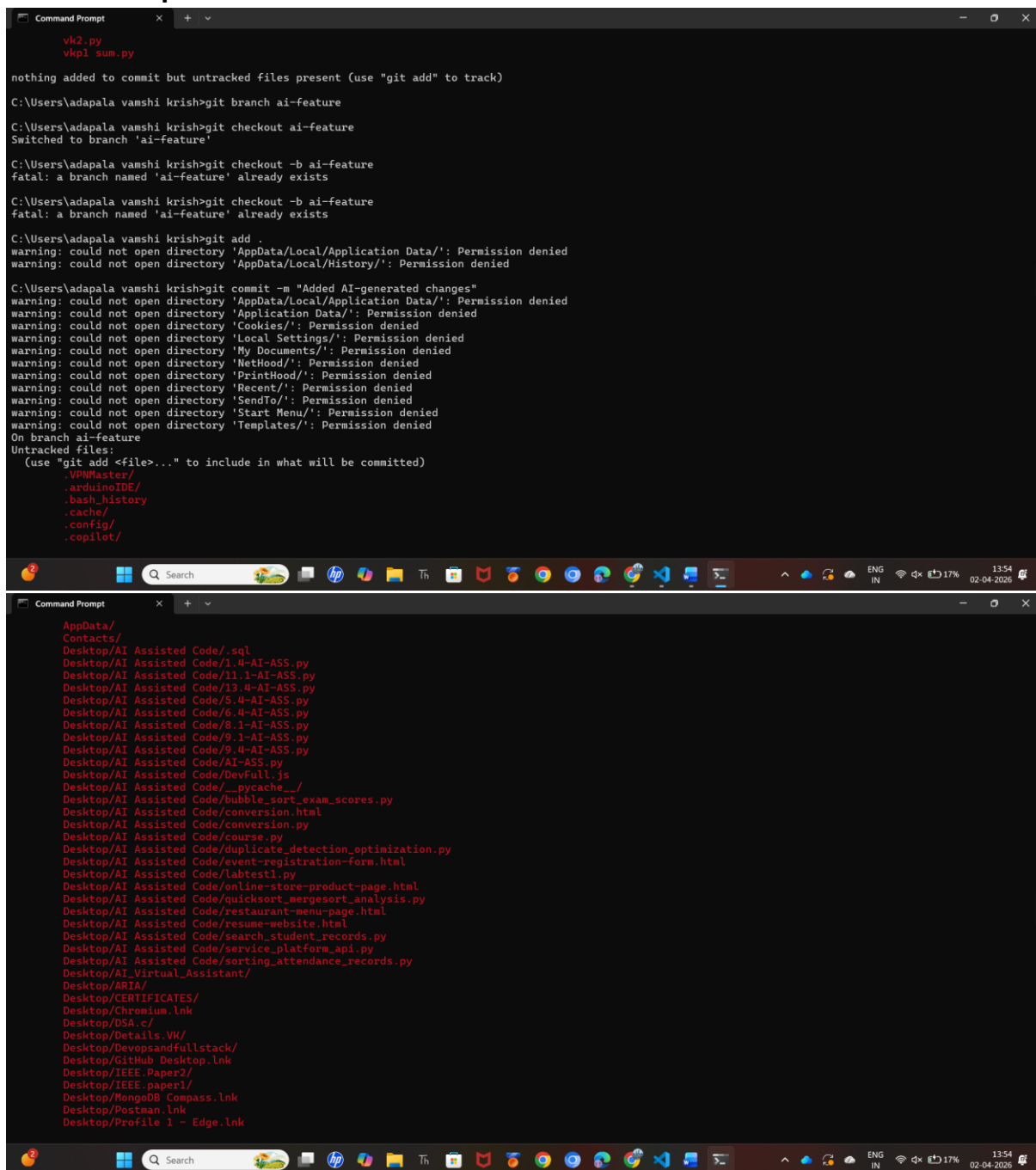
Prompt: Git Workflow Practice

Initialize a Git repository.

Create a new branch and add AI-generated changes.

Commit and merge changes into the main branch.

Code & Output:



```
vk2.py
vkpl sum.py

nothing added to commit but untracked files present (use "git add" to track)
C:\Users\adapala vamshi krish>git branch ai-feature
C:\Users\adapala vamshi krish>git checkout ai-feature
Switched to branch 'ai-feature'
C:\Users\adapala vamshi krish>git checkout -b ai-feature
fatal: a branch named 'ai-feature' already exists
C:\Users\adapala vamshi krish>git checkout -b ai-feature
fatal: a branch named 'ai-feature' already exists
C:\Users\adapala vamshi krish>git add .
warning: could not open directory 'AppData/Local/Application Data/': Permission denied
warning: could not open directory 'AppData/Local/History/': Permission denied
C:\Users\adapala vamshi krish>git commit -m "Added AI-generated changes"
warning: could not open directory 'AppData/Local/Application Data/': Permission denied
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch ai-feature
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .VPMaster/
        .arduinoIDE/
        .bash_history
        .cache/
        .config/
        .copilot/

C:\Users\adapala vamshi krish>git add .
AppData/
Contacts/
Desktop/AI Assisted Code/.sql
Desktop/AI Assisted Code/1.4-AI-ASS.py
Desktop/AI Assisted Code/11.1-AI-ASS.py
Desktop/AI Assisted Code/13.4-AI-ASS.py
Desktop/AI Assisted Code/5.4-AI-ASS.py
Desktop/AI Assisted Code/6.4-AI-ASS.py
Desktop/AI Assisted Code/8.1-AI-ASS.py
Desktop/AI Assisted Code/9.1-AI-ASS.py
Desktop/AI Assisted Code/9.4-AI-ASS.py
Desktop/AI Assisted Code/AI-ASS.py
Desktop/AI Assisted Code/DevFull.js
Desktop/AI Assisted Code/__pycache__/
Desktop/AI Assisted Code/bubble_sort_exam_scores.py
Desktop/AI Assisted Code/conversion.html
Desktop/AI Assisted Code/conversion.py
Desktop/AI Assisted Code/course.py
Desktop/AI Assisted Code/duplicate_detection_optimization.py
Desktop/AI Assisted Code/event-registration-form.html
Desktop/AI Assisted Code/labtest1.py
Desktop/AI Assisted Code/online-store-product-page.html
Desktop/AI Assisted Code/quick_sort_mergesort_analysis.py
Desktop/AI Assisted Code/restaurant-menu-page.html
Desktop/AI Assisted Code/resume-website.html
Desktop/AI Assisted Code/search_student_records.py
Desktop/AI Assisted Code/service_platform_api.py
Desktop/AI Assisted Code/sorting_attendance_records.py
Desktop/AI Virtual Assistant/
Desktop/ARIA/
Desktop/CERTIFICATES/
Desktop/Chromium.lnk
Desktop/DSA.c/
Desktop/Details.VK/
Desktop/Devopsandfullstack/
Desktop/GitHub Desktop.lnk
Desktop/IEEE.Paper2/
Desktop/IEEE.paper1/
Desktop/MongoDB Compass.lnk
Desktop/Postman.lnk
Desktop/Profile 1 - Edge.lnk
```

Observation:

Git Workflow Practice (Observation)

The Git workflow was successfully implemented by initializing a repository, creating a branch, and merging changes. AI-generated updates were integrated smoothly, demonstrating how version control supports safe and organized development practices.

TASK-4

Prompt: Branching and Merging with AI Support

Develop a program in the main branch.

Create a feature branch and modify using AI.

Merge and verify output.

Code & Output:

The top screenshot shows the initial code for 'InventoryManager' in the 'main' branch. The code includes a class 'InventoryManager' with methods for displaying items, validating names and stock, and analyzing improvements. The bottom screenshot shows the same code after a feature branch 'feature-update' was created, modified, and merged back to 'main'.

The bottom screenshot shows the code after a feature branch 'feature-update' was created, modified, and merged back to 'main'. The code is the same as the top screenshot, but the status bar at the bottom indicates the current branch is 'main'.

Observation:

Branching and Merging with AI Support (Observation)

The program developed in the main branch was enhanced in a feature branch using AI. The merge process was completed without conflicts, and the final output verified correctness. This task highlighted effective collaboration between AI assistance and version control workflows.

TASK-5

Prompt: AI for Commit Message Writing

Perform multiple code changes.

Use AI to generate commit messages.

Compare with manually written messages.

Code & Output:

The screenshot displays the VS Code interface with the `week11_task_d_inventory_manager.py` file open. The code includes a `InventoryManager` class with methods for validating item names and stock, and a `run_demo` function that demonstrates the inventory management system. The code is structured as follows:

```
class InventoryManager:
    def __init__(self):
        self.items = {}

    def validate_name(self, name):
        clean_name = name.strip()
        if not clean_name:
            raise ValueError("Item name cannot be empty.")
        return clean_name

    @staticmethod
    def validate_stock(stock):
        if not isinstance(stock, int):
            raise TypeError("Stock must be an integer.")
        if stock < 0:
            raise ValueError("Stock cannot be negative.")
        return stock

    def analyze_improvements():
        return [
            "Structure improvements:\n"
            "- Replaced plain dictionaries with an InventoryItem dataclass.\n"
            "- Used a dictionary keyed by item name for cleaner organization.\n"
            "- Added dedicated methods for validation, restocking, removal, and low-stock checks.\n"
            "Performance Improvements:\n"
            "- Search by name is now O(1) average case instead of linear scanning.\n"
            "- Updating stock is direct and avoids repeated list traversal.\n"
            "- Sorting happens only when displaying items, not during every update.\n"
            "- Category summaries are computed in a single pass over the stored items."
        ]

def run_demo():
    inventory = InventoryManager()
    inventory.add_item("Notebook", 12, "Stationery")
    inventory.add_item("Pen", 50, "Stationery")
    inventory.add_item("Marker", 8, "Stationery")
    inventory.add_item("USB Cable", 4, "Electronics")

    print("TASK D - AI-ENHANCED INVENTORY MANAGER")
    print("~" * 50)
    inventory.display_items()

    print("\nSearch example:")
    inventory.search("Marker")

    print("\nRestocking example:")
    inventory.restock("USB Cable", 10)

    print("\nRemoval example:")
    inventory.remove("Pen")

    print("\nLow-stock items (threshold = 10):")
    inventory.low_stock_items()

    print("\nCategory summary:")
    inventory.category_summary()
```

The right-hand pane shows the AI chat interface, which provides a detailed summary of the code changes and generates a commit message. The chat output includes:

- A summary of the code changes, highlighting the addition of a second layer of improvements (removal, category summaries, and a clearer demo flow) and the manual changes made to the commit message.
- A commit message generated by the AI, which is: `Improved Inventory Catalog`.
- A list of items in the inventory, categorized by type and stock level.
- A search example showing the results of a search for "Marker".
- A restocking example showing the results of a restock for "USB Cable".
- A removal example showing the results of a removal for "Pen".
- A low-stock items example showing the results of a low-stock check.
- A category summary example showing the results of a category summary.

Observation:

AI for Commit Message Writing (Observation)

AI-generated commit messages were clear, structured, and descriptive compared to manual ones. They improved readability and documentation of changes. This demonstrated that AI can standardize commit practices and enhance team collaboration.